Memory Segmentation

A compiled program's memory is divided into five segments: text, data, bss, heap, and stack. Each segment represents a special portion of memory that is set aside for a certain purpose.

The *text segment* is also sometimes called the *code segment*. This is where the assembled machine language instructions of the program are located. The execution of instructions in this segment is nonlinear, thanks to the aforementioned high-level control structures and functions, which compile into branch, jump, and call instructions in assembly language. As a program executes, the EIP is set to the first instruction in the text segment. The processor then follows an execution loop that does the following:

1. Reads the instruction that EIP is pointing to
2. Adds the byte length of the instruction to EIP
3. Executes the instruction that was read in step 1
4. Goes back to step 1

Sometimes the instruction will be a jump or a call instruction, which changes the EIP to a different address of memory. The processor doesn't care about the change, because it's expecting the execution to be nonlinear anyway. If EIP is changed in step 3, the processor will just go back to step 1 and read the instruction found at the address of whatever EIP was changed to.

Write permission is disabled in the text segment, as it is not used to store variables, only code. This prevents people from actually modifying the program code; any attempt to write to this segment of memory will cause the program to alert the user that something bad happened, and the program will be killed. Another advantage of this segment being read-only is

that it can be shared among different copies of the program, allowing multiple executions of the program at the same time without any problems. It should also be noted that this memory segment has a fixed size, since nothing ever changes in it.

The data and bss segments are used to store global and static program variables. The *data segment* is filled with the initialized global and static variables, while the *bss segment* is filled with their uninitialized counterparts. Although these segments are writable, they also have a fixed size. Remember that global variables persist, despite the functional context (like the variable `j` in the previous examples). Both global and static variables are able to persist because they are stored in their own memory segments.

The *heap segment* is a segment of memory a programmer can directly control. Blocks of memory in this segment can be allocated and used for whatever the programmer might need. One notable point about the heap segment is that it isn't of fixed size, so it can grow larger or smaller as needed. All of the memory within the heap is managed by allocator and deallocator algorithms, which respectively reserve a region of memory in the heap for use and remove reservations to allow that portion of memory to be reused for later reservations. The heap will grow and shrink depending on how much memory is reserved for use. This means a programmer using the heap allocation functions can reserve and free memory on the fly. The growth of the heap moves downward toward higher memory addresses.

The *stack segment* also has variable size and is used as a temporary scratch pad to store local function variables and context during function calls. This is what GDB's backtrace command looks at. When a program calls a function, that function will have its own set of passed variables, and the function's code will be at a different memory location in the text (or code) segment. Since the context and the EIP must change when a function is called, the stack is used to remember all of the passed variables, the location the EIP

should return to after the function is finished, and all the local variables used by that function. All of this information is stored together on the stack in what is collectively called a *stack frame*. The stack contains many stack frames.

In general computer science terms, a *stack* is an abstract data structure that is used frequently. It has *first-in, last-out (FILO) ordering*, which means the first item that is put into a stack is the last item to come out of it. Think of it as putting beads on a piece of string that has a knot on one end—you can't get the first bead off until you have removed all the other beads. When an item is placed into a stack, it's known as *pushing*, and when an item is removed from a stack, it's called *popping*.

As the name implies, the stack segment of memory is, in fact, a stack data structure, which contains stack frames. The ESP register is used to keep track of the address of the end of the stack, which is constantly changing as items are pushed into and popped off of it. Since this is very dynamic behavior, it makes sense that the stack is also not of a fixed size. Opposite to the dynamic growth of the heap, as the stack changes in size, it grows upward in a visual listing of memory, toward lower memory addresses.

The FILO nature of a stack might seem odd, but since the stack is used to store context, it's very useful. When a function is called, several things are pushed to the stack together in a *stack frame*. The EBP register—sometimes called the *frame pointer (FP)* or *local base (LB) pointer*— is used to reference local function variables in the current stack frame. Each stack frame contains the parameters to the function, its local variables, and two pointers that are necessary to put things back the way they were: the saved frame pointer (SFP) and the return address. The *SFP* is used to restore EBP to its previous value, and the *return address* is used to restore EIP to the next instruction found after the function call. This restores the functional context of the previous stack frame.

The following stack_example.c code has two functions: `main()`

and `test_function()`.

Memory Segmentation

stack_example.c

```
void test_function(int a, int b, int c, int d) {
   int flag;
   char buffer[10];

   flag = 31337;
   buffer[0] = 'A';
}

int main() {
   test_function(1, 2, 3, 4);
}
```

This program first declares a test function that has four arguments, which are all declared as integers: `a`, `b`, `c`, and `d`. The local variables for the function include a single character called `flag` and a 10-character buffer called `buffer`. The memory for these variables is in the stack segment, while the machine instructions for the function's code is stored in the text segment. After compiling the program, its inner workings can be examined with GDB. The following output shows the disassembled machine instructions for `main()` and `test_function()`. The `main()` function starts at `0x08048357` and `test_function()`starts at `0x08048344`. The first few instructions of each function (shown in bold below) set up the stack frame. These instructions are collectively called the *procedure prologue* or *function prologue*. They save the frame pointer on the stack, and they save stack memory for the local function variables. Sometimes the function prologue will handle some stack alignment as well. The exact prologue instructions will vary greatly depending on the compiler and compiler options, but in general these instructions build the stack frame.

```
reader@hacking:~/booksrc $ gcc -g stack_example.c
reader@hacking:~/booksrc $ gdb -q ./a.out
Using host libthread_db library
```

```
"/lib/tls/i686/cmov/libthread_db.so.1".
(gdb) disass main
Dump of assembler code for function main():
0x08048357 <main+0>:    push   ebp
0x08048358 <main+1>:    mov    ebp,esp
0x0804835a <main+3>:    sub    esp,0x18
0x0804835d <main+6>:    and    esp,0xfffffff0
0x08048360 <main+9>:    mov    eax,0x0
0x08048365 <main+14>:   sub    esp,eax
0x08048367 <main+16>:   mov    DWORD PTR [esp+12],0x4
0x0804836f <main+24>:   mov    DWORD PTR [esp+8],0x3
0x08048377 <main+32>:   mov    DWORD PTR [esp+4],0x2
0x0804837f <main+40>:   mov    DWORD PTR [esp],0x1
0x08048386 <main+47>:   call   0x8048344 <test_function>
0x0804838b <main+52>:   leave
0x0804838c <main+53>:   ret
End of assembler dump
(gdb) disass test_function()
Dump of assembler code for function test_function:
0x08048344 <test_function+0>:   push   ebp
0x08048345 <test_function+1>:   mov    ebp,esp
0x08048347 <test_function+3>:   sub    esp,0x28
0x0804834a <test_function+6>:   mov    DWORD PTR [ebp-12],0x7a69
0x08048351 <test_function+13>:  mov    BYTE PTR [ebp-40],0x41
0x08048355 <test_function+17>:  leave
0x08048356 <test_function+18>:  ret
End of assembler dump
(gdb)
```

When the program is run, the `main()` function is called, which simply calls `test_function()`.

When the `test_function()` is called from the `main()` function, the various values are pushed to the stack to create the start of the stack frame as follows. When `test_function()` is called, the function arguments are pushed onto the stack in reverse order (since it's FILO). The arguments for the function are 1, 2, 3, and 4, so the subsequent push instructions push 4, 3, 2, and finally 1 onto the stack. These values correspond to the variables `d`, `c`, `b`, and `a` in the function. The instructions that put these values on the stack are shown in bold in the `main()` function's disassembly below.

```
(gdb) disass main
Dump of assembler code for function main:
0x08048357 <main+0>:    push   ebp
0x08048358 <main+1>:    mov    ebp,esp
0x0804835a <main+3>:    sub    esp,0x18
```

```
0x0804835d <main+6>:    and    esp,0xfffffff0
0x08048360 <main+9>:    mov    eax,0x0
0x08048365 <main+14>:   sub    esp,eax
0x08048367 <main+16>:   mov    DWORD PTR [esp+12],0x4
0x0804836f <main+24>:   mov    DWORD PTR [esp+8],0x3
0x08048377 <main+32>:   mov    DWORD PTR [esp+4],0x2
0x0804837f <main+40>:   mov    DWORD PTR [esp],0x1
0x08048386 <main+47>:   call   0x8048344 <test_function>
0x0804838b <main+52>:   leave
0x0804838c <main+53>:   ret
End of assembler dump
(gdb)
```

Next, when the assembly call instruction is executed, the return address is pushed onto the stack and the execution flow jumps to the start of `test_function()` at `0x08048344`. The return address value will be the location of the instruction following the current EIP—specifically, the value stored during step 3 of the previously mentioned execution loop. In this case, the return address would point to the leave instruction in `main()` at `0x0804838b`.

The call instruction both stores the return address on the stack and jumps EIP to the beginning of `test_function()`, so `test_function()`'s procedure prologue instructions finish building the stack frame. In this step, the current value of EBP is pushed to the stack. This value is called the saved frame pointer (SFP) and is later used to restore EBP back to its original state. The current value of ESP is then copied into EBP to set the new frame pointer. This frame pointer is used to reference the local variables of the function (`flag` and `buffer`). Memory is saved for these variables by subtracting fromESP. In the end, the stack frame looks something like this:

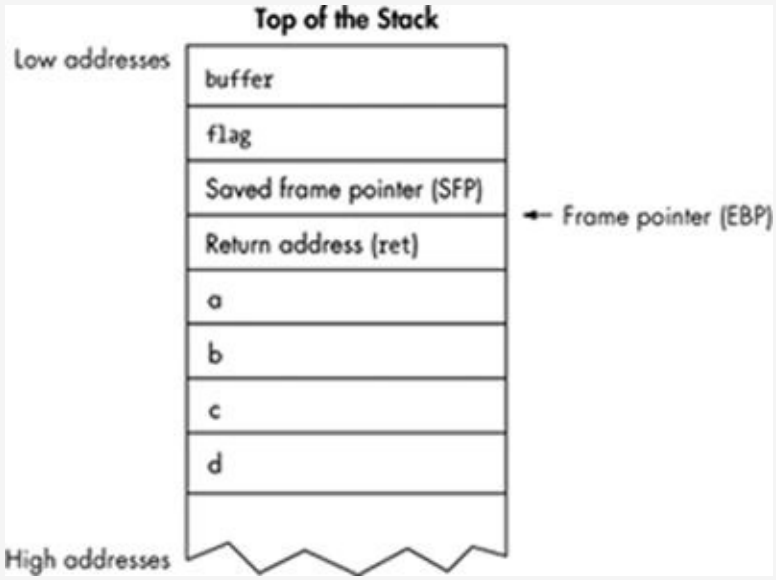

Figure 0x200-1.

We can watch the stack frame construction on the stack using GDB. In the following output, a breakpoint is set in `main()` before the call to `test_function()`and also at the beginning of `test_function()`. GDB will put the first breakpoint before the function arguments are pushed to the stack, and the second breakpoint after `test_function()`'s procedure prologue. When the program is run, execution stops at the breakpoint, where the register's ESP (stack pointer), EBP (frame pointer), and EIP (execution pointer) are examined.

```
(gdb) list main
4
5          flag = 31337;
6          buffer[0] = 'A';
7       }
8
9       int main() {
10         test_function(1, 2, 3, 4);
11      }
(gdb) break 10
Breakpoint 1 at 0x8048367: file stack_example.c, line 10.
(gdb) break test_function
Breakpoint 2 at 0x804834a: file stack_example.c, line 5.
(gdb) run
Starting program: /home/reader/booksrc/a.out

Breakpoint 1, main () at stack_example.c:10
10         test_function(1, 2, 3, 4);
```

```
(gdb) i r esp ebp eip
esp            0xbffff7f0       0xbffff7f0
ebp            0xbffff808       0xbffff808
eip            0x8048367        0x8048367 <main+16>
(gdb) x/5i $eip
0x8048367 <main+16>:    mov    DWORD PTR [esp+12],0x4
0x804836f <main+24>:    mov    DWORD PTR [esp+8],0x3
0x8048377 <main+32>:    mov    DWORD PTR [esp+4],0x2
0x804837f <main+40>:    mov    DWORD PTR [esp],0x1
0x8048386 <main+47>:    call   0x8048344 <test_function>
(gdb)
```

This breakpoint is right before the stack frame for the `test_function()` call is created. This means the bottom of this new stack frame is at the current value of ESP, `0xbffff7f0`. The next breakpoint is right after the procedure prologue for `test_function()`, so continuing will build the stack frame. The output below shows similar information at the second breakpoint. The local variables (`flag` and `buffer`) are referenced relative to the frame pointer (EBP).

```
(gdb) cont
Continuing.

Breakpoint 2, test_function (a=1, b=2, c=3, d=4) at stack_example.c:5
5          flag = 31337;
(gdb) i r esp ebp eip
esp            0xbffff7c0       0xbffff7c0
ebp            0xbffff7e8       0xbffff7e8
eip            0x804834a        0x804834a <test_function+6>
(gdb) disass test_function
Dump of assembler code for function test_function:
0x08048344 <test_function+0>:   push   ebp
0x08048345 <test_function+1>:   mov    ebp,esp
0x08048347 <test_function+3>:   sub    esp,0x28
0x0804834a <test_function+6>:   mov    DWORD PTR [ebp-12],0x7a69
0x08048351 <test_function+13>:  mov    BYTE PTR [ebp-40],0x41
0x08048355 <test_function+17>:  leave
0x08048356 <test_function+18>:  ret
End of assembler dump.
(gdb) print $ebp-12
$1 = (void *) 0xbffff7dc
(gdb) print $ebp-40
$2 = (void *) 0xbffff7c0
(gdb) x/16xw $esp
0xbffff7c0:    ❶ 0x00000000      0x08049548      0xbffff7d8
0x08048249
```

```
0xbffff7d0:     0xb7f9f729      0xb7fd6ff4      0xbffff808
0x080483b9
0xbffff7e0:     0xb7fd6ff4      ❷0xbffff89c     ❸0xbffff808
❹0x0804838b
0xbffff7f0:     ❺0x00000001     0x00000002      0x00000003      0x00000004
(gdb)
```

The stack frame is shown on the stack at the end. The four arguments to the function can be seen at the bottom of the stack frame (❺), with the return address found directly on top (❹). Above that is the saved frame pointer of `0xbffff808` (❸), which is what EBP was in the previous stack frame. The rest of the memory is saved for the local stack variables: `flag` and `buffer`. Calculating their relative addresses to EBP show their exact locations in the stack frame. Memory for the `flag` variable is shown at ❷ and memory for the buffer variable is shown at ❶. The extra space in the stack frame is just padding.

After the execution finishes, the entire stack frame is popped off of the stack, and the EIP is set to the return address so the program can continue execution. If another function was called within the function, another stack frame would be pushed onto the stack, and so on. As each function ends, its stack frame is popped off of the stack so execution can be returned to the previous function. This behavior is the reason this segment of memory is organized in a FILO data structure.

The various segments of memory are arranged in the order they were presented, from the lower memory addresses to the higher memory addresses. Since most people are familiar with seeing numbered lists that count downward, the smaller memory addresses are shown at the top. Some texts have this reversed, which can be very confusing; so for this book, smaller memory addresses are always shown at the top. Most debuggers also display memory in this style, with the smaller memory addresses at the top and the higher ones at the bottom.

Since the heap and the stack are both dynamic, they both grow in different directions toward each other. This minimizes

wasted space, allowing the stack to be larger if the heap is small and vice versa.

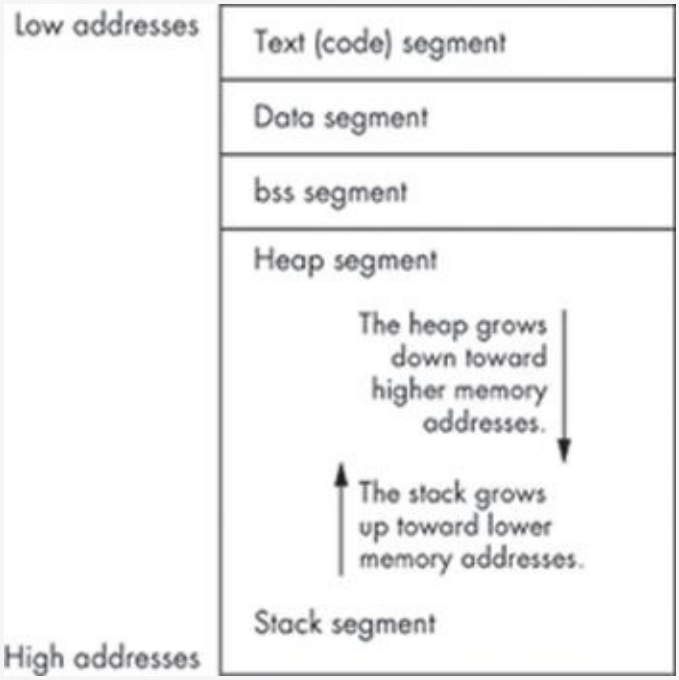

Figure 0x200-2.

Memory Segments in C

In C, as in other compiled languages, the compiled code goes into the text segment, while the variables reside in the remaining segments. Exactly which memory segment a variable will be stored in depends on how the variable is defined. Variables that are defined outside of any functions are considered to be global. The `static` keyword can also be prepended to any variable declaration to make the variable static. If static or global variables are initialized with data, they are stored in the data memory segment; otherwise, these variables are put in the bss memory segment. Memory on the heap memory segment must first be allocated using a memory allocation function called `malloc()`. Usually, pointers are used to reference memory on the heap. Finally, the remaining function variables are stored in the stack memory segment. Since the stack can contain many different stack frames, stack

variables can maintain uniqueness within different functional contexts. The memory_segments.c program will help explain these concepts in C.

memory_segments.c

```
#include <stdio.h>

int global_var;

int global_initialized_var = 5;

void function() {  // This is just a demo function.
   int stack_var; // Notice this variable has the same name as the
one in main().

   printf("the function's stack_var is at address 0x%08x\n",
&stack_var);
}


int main() {
   int stack_var; // Same name as the variable in function()
   static int static_initialized_var = 5;
   static int static_var;
   int *heap_var_ptr;

   heap_var_ptr = (int *) malloc(4);

   // These variables are in the data segment.
   printf("global_initialized_var is at address 0x%08x\n",
&global_initialized_var);
   printf("static_initialized_var is at address 0x%08x\n\n",
&static_initialized_var);

   // These variables are in the bss segment.
   printf("static_var is at address 0x%08x\n", &static_var);
   printf("global_var is at address 0x%08x\n\n", &global_var);

   // This variable is in the heap segment.
   printf("heap_var is at address 0x%08x\n\n", heap_var_ptr);

   // These variables are in the stack segment.
   printf("stack_var is at address 0x%08x\n", &stack_var);
   function();
}
```

Most of this code is fairly self-explanatory because of the

descriptive variable names. The global and static variables are declared as described earlier, and initialized counterparts are also declared. The stack variable is declared both in `main()` and in `function()` to showcase the effect of functional contexts. The heap variable is actually declared as an integer pointer, which will point to memory allocated on the heap memory segment. The `malloc()`function is called to allocate four bytes on the heap. Since the newly allocated memory could be of any data type, the `malloc()` function returns a void pointer, which needs to be typecast into an integer pointer.

```
reader@hacking:~/booksrc $ gcc memory_segments.c
reader@hacking:~/booksrc $ ./a.out
global_initialized_var is at address 0x080497ec
static_initialized_var is at address 0x080497f0

static_var is at address 0x080497f8
global_var is at address 0x080497fc

heap_var is at address 0x0804a008

stack_var is at address 0xbffff834
the function's stack_var is at address 0xbffff814
reader@hack ing:~/booksrc $
```

The first two initialized variables have the lowest memory addresses, since they are located in the data memory segment. The next two variables, `static_var` and `global_var`, are stored in the bss memory segment, since they aren't initialized. These memory addresses are slightly larger than the previous variables' addresses, since the bss segment is located below the data segment. Since both of these memory segments have a fixed size after compilation, there is little wasted space, and the addresses aren't very far apart.

The heap variable is stored in space allocated on the heap segment, which is located just below the bss segment. Remember that memory in this segment isn't fixed, and more space can be dynamically allocated later. Finally, the last two `stack_vars` have very large memory addresses, since they are located in the stack segment. Memory in the stack isn't fixed, either; however, this memory starts at the bottom and grows

backward toward the heap segment. This allows both memory segments to be dynamic without wasting space in memory. The first `stack_var` in the `main()` function's context is stored in the stack segment within a stack frame. The second `stack_var` in `function()` has its own unique context, so that variable is stored within a different stack frame in the stack segment. When `function()` is called near the end of the program, a new stack frame is created to store (among other things) the `stack_var` for `function()`'s context. Since the stack grows back up toward the heap segment with each new stack frame, the memory address for the second `stack_var`(`0xbffff814`) is smaller than the address for the first `stack_var` (`0xbffff834`) found within `main()`'s context.

Using the Heap

Using the other memory segments is simply a matter of how you declare variables. However, using the heap requires a bit more effort. As previously demonstrated, allocating memory on the heap is done using the `malloc()`function. This function accepts a size as its only argument and reserves that much space in the heap segment, returning the address to the start of this memory as a void pointer. If the `malloc()` function can't allocate memory for some reason, it will simply return a NULL pointer with a value of 0. The corresponding deallocation function is `free()`. This function accepts a pointer as its only argument and frees that memory space on the heap so it can be used again later. These relatively simple functions are demonstrated in heap_example.c.

heap_example.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
   char *char_ptr;  // A char pointer
```

```
   int *int_ptr;    // An integer pointer
   int mem_size;

   if (argc < 2)     // If there aren't command-line arguments,
      mem_size = 50; // use 50 as the default value.
   else
      mem_size = atoi(argv[1]);

   printf("\t[+] allocating %d bytes of memory on the heap for
char_ptr\n", mem_size);
   char_ptr = (char *) malloc(mem_size); // Allocating heap memory

   if(char_ptr == NULL) {  // Error checking, in case malloc() fails
      fprintf(stderr, "Error: could not allocate heap memory.\n");
      exit(-1);
   }

   strcpy(char_ptr, "This is memory is located on the heap.");
   printf("char_ptr (%p) --> '%s'\n", char_ptr, char_ptr);

   printf("\t[+] allocating 12 bytes of memory on the heap for
int_ptr\n");
   int_ptr = (int *) malloc(12); // Allocated heap memory again

   if(int_ptr == NULL) {  // Error checking, in case malloc() fails
      fprintf(stderr, "Error: could not allocate heap memory.\n");
      exit(-1);
   }

   *int_ptr = 31337; // Put the value of 31337 where int_ptr is
pointing.
   printf("int_ptr (%p) --> %d\n", int_ptr, *int_ptr);


   printf("\t[-] freeing char_ptr's heap memory...\n");
   free(char_ptr); // Freeing heap memory

   printf("\t[+] allocating another 15 bytes for char_ptr\n");
   char_ptr = (char *) malloc(15); // Allocating more heap memory

   if(char_ptr == NULL) {  // Error checking, in case malloc() fails
      fprintf(stderr, "Error: could not allocate heap memory.\n");
      exit(-1);
   }

   strcpy(char_ptr, "new memory");
   printf("char_ptr (%p) --> '%s'\n", char_ptr, char_ptr);

   printf("\t[-] freeing int_ptr's heap memory...\n");
```

```
    free(int_ptr); // Freeing heap memory
    printf("\t[-] freeing char_ptr's heap memory...\n");
    free(char_ptr); // Freeing the other block of heap memory
}
```

This program accepts a command-line argument for the size of the first memory allocation, with a default value of 50. Then it uses the `malloc()` and `free()` functions to allocate and deallocate memory on the heap. There are plenty of `printf()` statements to debug what is actually happening when the program is executed. Since `malloc()` doesn't know what type of memory it's allocating, it returns a void pointer to the newly allocated heap memory, which must be typecast into the appropriate type. After every `malloc()` call, there is an error-checking block that checks whether or not the allocation failed. If the allocation fails and the pointer is NULL, `fprintf()` is used to print an error message to standard error and the program exits. The `fprintf()` function is very similar to `printf()`; however, its first argument is `stderr`, which is a standard filestream meant for displaying errors. This function will be explained more later, but for now, it's just used as a way to properly display an error. The rest of the program is pretty straightforward.

```
reader@hacking:~/booksrc $ gcc -o heap_example heap_example.c
reader@hacking:~/booksrc $ ./heap_example
        [+] allocating 50 bytes of memory on the heap for char_ptr
char_ptr (0x804a008) --> 'This is memory is located on the heap.'
        [+] allocating 12 bytes of memory on the heap for int_ptr
int_ptr (0x804a040) --> 31337
        [-] freeing char_ptr's heap memory...
        [+] allocating another 15 bytes for char_ptr
char_ptr (0x804a050) --> 'new memory'
        [-] freeing int_ptr's heap memory...
        [-] freeing char_ptr's heap memory...
reader@hacking:~/booksrc $
```

In the preceding output, notice that each block of memory has an incrementally higher memory address in the heap. Even though the first 50 bytes were deallocated, when 15 more bytes are requested, they are put after the 12 bytes allocated for the `int_ptr`. The heap allocation functions control this behavior, which can be explored by changing the size of the initial

memory allocation.

```
reader@hacking:~/booksrc $ ./heap_example 100
        [+] allocating 100 bytes of memory on the heap for char_ptr
char_ptr (0x804a008) --> 'This is memory is located on the heap.'
        [+] allocating 12 bytes of memory on the heap for int_ptr
int_ptr (0x804a070) --> 31337
        [-] freeing char_ptr's heap memory...
        [+] allocating another 15 bytes for char_ptr
char_ptr (0x804a008) --> 'new memory'
        [-] freeing int_ptr's heap memory...
        [-] freeing char_ptr's heap memory...
reader@hacking:~/booksrc $
```

If a larger block of memory is allocated and then deallocated, the final 15-byte allocation will occur in that freed memory space, instead. By experimenting with different values, you can figure out exactly when the allocation function chooses to reclaim freed space for new allocations. Often, simple informative `printf()` statements and a little experimentation can reveal many things about the underlying system.

Error-Checked malloc()

In heap_example.c, there were several error checks for the `malloc()` calls. Even though the `malloc()` calls never failed, it's important to handle all potential cases when coding in C. But with multiple `malloc()` calls, this error-checking code needs to appear in multiple places. This usually makes the code look sloppy, and it's inconvenient if changes need to be made to the error-checking code or if new `malloc()` calls are needed. Since all the error-checking code is basically the same for every `malloc()` call, this is a perfect place to use a function instead of repeating the same instructions in multiple places. Take a look at errorchecked_heap.c for an example.

errorchecked_heap.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void *errorchecked_malloc(unsigned int); // Function prototype for
errorchecked_malloc()

int main(int argc, char *argv[]) {
   char *char_ptr;  // A char pointer
   int *int_ptr;    // An integer pointer
   int mem_size;

   if (argc < 2)     // If there aren't command-line arguments,
      mem_size = 50; // use 50 as the default value.
   else
      mem_size = atoi(argv[1]);

   printf("\t[+] allocating %d bytes of memory on the heap for
char_ptr\n", mem_size);
   char_ptr = (char *) errorchecked_malloc(mem_size); // Allocating
heap memory

   strcpy(char_ptr, "This is memory is located on the heap.");
   printf("char_ptr (%p) --> '%s'\n", char_ptr, char_ptr);
   printf("\t[+] allocating 12 bytes of memory on the heap for
int_ptr\n");
   int_ptr = (int *) errorchecked_malloc(12); // Allocated heap
memory again

   *int_ptr = 31337; // Put the value of 31337 where int_ptr is
pointing.
   printf("int_ptr (%p) --> %d\n", int_ptr, *int_ptr);

   printf("\t[-] freeing char_ptr's heap memory...\n");
   free(char_ptr); // Freeing heap memory

   printf("\t[+] allocating another 15 bytes for char_ptr\n");
   char_ptr = (char *) errorchecked_malloc(15); // Allocating more
heap memory

   strcpy(char_ptr, "new memory");
   printf("char_ptr (%p) --> '%s'\n", char_ptr, char_ptr);

   printf("\t[-] freeing int_ptr's heap memory...\n");
   free(int_ptr); // Freeing heap memory
   printf("\t[-] freeing char_ptr's heap memory...\n");
   free(char_ptr); // Freeing the other block of heap memory
}

void *errorchecked_malloc(unsigned int size) { // An error-checked
malloc() function
   void *ptr;
```

```
    ptr = malloc(size);
    if(ptr == NULL) {
       fprintf(stderr, "Error: could not allocate heap memory.\n");
       exit(-1);
    }
    return ptr;
}
```

The errorchecked_heap.c program is basically equivalent to the previous heap_example.c code, except the heap memory allocation and error checking has been gathered into a single function. The first line of code [`void *errorchecked_malloc(unsigned int);`] is the function prototype. This lets the compiler know that there will be a function called `errorchecked_malloc()` that expects a single, unsigned integer argument and returns a `void` pointer. The actual function can then be anywhere; in this case it is after the `main()` function. The function itself is quite simple; it just accepts the size in bytes to allocate and attempts to allocate that much memory using `malloc()`. If the allocation fails, the error-checking code displays an error and the program exits; otherwise, it returns the pointer to the newly allocated heap memory. This way, the custom `errorchecked_malloc()` function can be used in place of a normal `malloc()`, eliminating the need for repetitious error checking afterward. This should begin to highlight the usefulness of programming with functions.